

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

дисциплина: Архитектура компьютера _

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Базовые сведения о Markdown

2.1.1 Параграф

2.1.2 Заголовки

2.1.3 Выделение текста

2.1.4 Ссылки

2.1.5 Маркированные

2.2 Оформление формул в Markdown

2.3. Оформление изображений в Markdown

2.3.1 Локальные изображения

2.3.2 Внешние изображение

Выполнения лабораторной работы

Выполнение самостоятельной работы

Выводы

Список литературы

Цель работы

Целью работы является освоение процедуры оформления отчетов с помощью легковесного языка разметки Markdown

Теоретическое введение

2. 1 Базовые сведения о Markdown

Markdown — это простой язык разметки. Его создал John Gruber в 2004 году. Markdown помогает писать текст, который потом можно превратить в HTML. Синтаксис очень простой: заголовки пишут с #, жирный текст — **текст**, списки — - или *, ссылки — [текст](адрес). Markdown используют на GitHub, GitLab, Reddit и для технической документации.

Markdown удобен для работы с Git, потому что файлы .md легко сравнивать. Есть разные версии Markdown, например GitHub Flavored Markdown, где можно делать таблицы и красивый код. Основные источники — сайт Daring Fireball и документация GitHub.

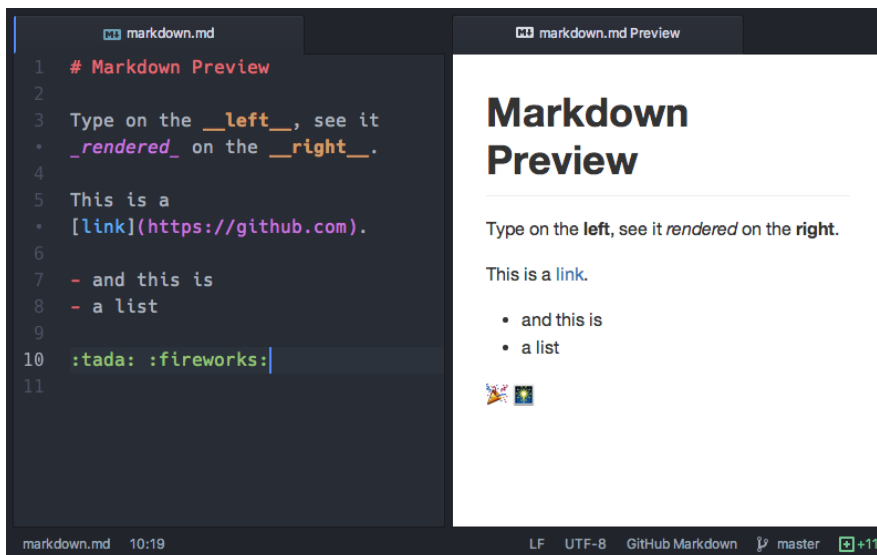


2.1.1 Параграф

В Markdown параграф — это текст, который идёт до пустой строки. Чтобы создать параграф, просто пишите текст. Когда вы хотите начать новый параграф, оставьте пустую строку (нажмите Enter два раза). Это очень похоже на то, как мы пишем обычные тексты в письмах или сообщениях.

Если нужно перейти на новую строку внутри одного параграфа (например, в стихотворении или адресе), поставьте два пробела в конце строки и нажмите Enter один раз. Это называется "жёсткий перенос". Markdown создан так, чтобы текст было легко читать даже без специальных программ. Когда вы загружаете файл .md на GitHub или другой сайт, программа сама красиво оформляет параграфы — добавляет отступы и правильные переносы.

Некоторые программы могут по-разному понимать пробелы и пустые строки, поэтому лучше следовать стандарту CommonMark. Так ваш текст будет везде выглядеть одинаково.

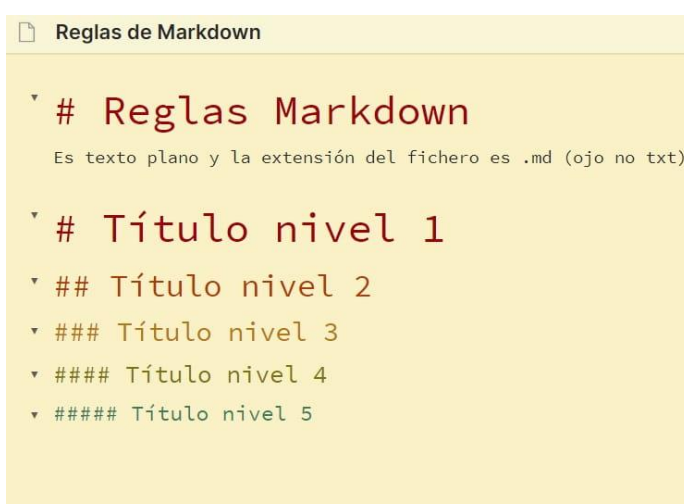


2.1.2 Заголовки

В Markdown заголовки создаются с помощью символа решётки (#) в начале строки. Количество решёток показывает уровень

заголовка: # — это заголовок первого уровня (самый крупный), ## — второго, и так можно использовать до шести решёток (#####). Между последней решёткой и текстом лучше ставить пробел — так Markdown правильно поймёт структуру. Есть и старый способ создания заголовков первого и второго уровней с помощью подчёркивания знаками = или -, но сейчас он используется реже.

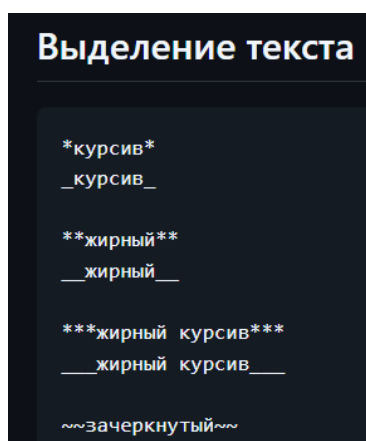
Заголовки в Markdown не только делают текст красивым, но и помогают создавать оглавления. На GitHub и в других системах, если нажать на заголовок, можно получить прямую ссылку на этот раздел. Это очень удобно для больших документов, инструкций или README-файлов в проектах. Заголовки также улучшают доступность документа для программ чтения с экрана и помогают поисковым системам понимать структуру контента.



2.1.3 Выделение текста

В Markdown выделение текста делается с помощью звёздочек (*) или подчёркиваний (_). Чтобы сделать текст курсивом, нужно поставить одну звёздочку или одно подчёркивание с двух сторон слова: `*текст*` или `_текст_`. Для жирного текста используются две звёздочки или два подчёркивания: `**текст**` или `__текст__`. Если нужно сделать текст и жирным, и курсивом одновременно, можно использовать три звёздочки или три подчёркивания: `***текст***` или `___текст___`.

Хотя оба символа работают, звёздочки (*) обычно рекомендуются, потому что их легче читать в исходном коде, и они лучше поддерживаются разными программами. Важно, что даже с этими символами текст остаётся понятным в обычном редакторе — это главная идея Markdown. В расширенных версиях, таких как GitHub Markdown, также можно зачёркивать текст (`~~зачеркнутый~~`) и выделять строчный код (``код``) для технических терминов или команд.



2.1.4 Ссылки

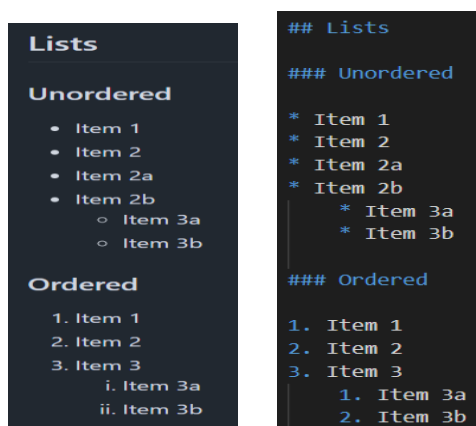
В Markdown ссылки создаются с помощью простой структуры: `[видимый текст](адрес)`. Когда документ обрабатывается, это превращается в обычную кликабельную ссылку в HTML. Например, `[Google](https://google.com)` станет ссылкой на Google. Для удобства в длинных документах можно использовать ссылку-ссылку: сначала пишете `[текст][метка]`, а потом в любом месте документа (обычно в конце) определяете `[метка]: https://адрес "подсказка"`. Это помогает не повторять длинные URL много раз.

Markdown также позволяет делать внутренние ссылки на заголовки внутри того же документа. Для этого используется `[текст](#id-заголовка)`, где ID создаётся автоматически из текста заголовка (пробелы заменяются на дефисы, убираются знаки препинания). В

современных версиях Markdown, таких как на GitHub, ссылки могут создаваться автоматически: если написать обычный URL или email в угловых скобках (<https://example.com>), он станет ссылкой без дополнительных символов.

2.1.5 Маркированные

Маркированные (неупорядоченные) списки в Markdown создаются с помощью символов -, * или +, за которыми следует пробел. Каждый элемент списка начинается с новой строки. Для создания вложенных списков (подсписков) необходимо добавить отступ в два или четыре пробела перед маркером элемента. Markdown автоматически преобразует эти символы в корректные HTML-теги и при рендеринге, обеспечивая чистую и читаемую структуру как в исходном коде, так и в конечном отформатированном документе. Это один из самых простых и интуитивно понятных инструментов для организации перечней пунктов, мыслей или элементов без определенного порядка.



2.2 Оформление формул в Markdown

Для работы с математическими формулами в Markdown стал стандартом синтаксис LaTeX, особенно в диалектах, поддерживающих расширения, такие как Markdown с MathJax или KaTeX. Формулы могут быть встроенными (inline) и вынесенными (display). Встроенные формулы заключаются в одинарные знаки доллара (\$...\$), например, $E = mc^2$, и отображаются в потоке текста. Вынесенные, или блочные, формулы, которые центрируются на отдельной строке, заключаются в двойные знаки доллара (\$\$...\$\$). Это позволяет набирать сложные математические выражения, используя мощь LaTeX: дроби ($\frac{}{}$), интегралы ($\int$), греческие буквы (α , β) и матрицы. Поддержка этой функциональности зависит от конкретного движка рендеринга (например, GitHub поддерживает MathJax в .md-файлах через \$\$, а многие статические генераторы сайтов требуют явного подключения библиотек).

Compute $f(x) = x^2 + 2$ if $x=2$.



Compute $f(x) = x^2 + 2$ if $x = 2$.

Newton postulated that $\vec{F} = m \vec{a}$.



Newton postulated that $\vec{F} = m\vec{a}$.

2.3. Оформление изображений в Markdown

Markdown позволяет встраивать изображения с помощью интуитивного синтаксиса, похожего на создание ссылок. Базовая структура выглядит так: `![Альтернативный текст](URL_изображения "Необязательный заголовок")`. Символ восклицательного знака (!) в начале указывает на то, что это элемент изображения, а не просто ссылка. Альтернативный текст (alt text) — это критически важный элемент доступности, который описывает изображение для пользователей скринридеров и отображается, если картинка не может быть загружена. URL может быть абсолютным путем к файлу в интернете или относительным путем к файлу в структуре локального проекта. Заголовок (title), заключаемый в кавычки, является необязательным и обычно отображается в виде всплывающей подсказки при наведении курсора. Для более сложного управления (изменение размера, выравнивание) часто приходится прибегать к прямому встраиванию HTML-тега `<img>` с нужными атрибутами, что поддерживается большинством процессоров Markdown.

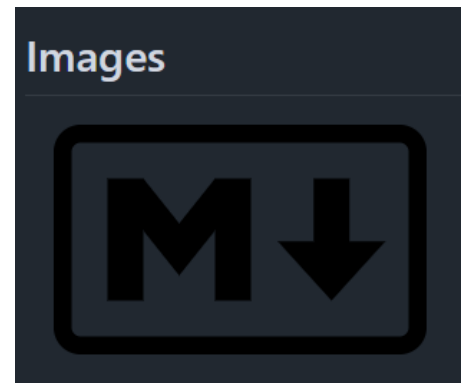
2.3.1 Локальные изображения

Для вставки локальных изображений в Markdown используется тот же базовый синтаксис, но вместо веб-URL указывается относительный или абсолютный путь к файлу в вашей файловой системе. Относительный путь (например, `![Моя схема](./diagrams/schema.png)`) является предпочтительным, так как обеспечивает переносимость проекта: если папка с документом и изображением перемещается вместе, ссылка не нарушается. Ключевой момент — движок рендеринга (браузер, движок GitHub, статический генератор сайтов) должен иметь доступ к этому файлу по указанному пути. При работе с локальными превьюверрами или некоторыми IDE изображения могут отображаться сразу. Однако для публикации в вебе (например, на GitHub Pages или в блоге) часто требуется,

чтобы изображения были частью репозитория и имели корректный публичный путь. Для управления размером или стилем локальных изображений стандартный Markdown не предоставляет возможностей, поэтому на практике часто используется гибридный подход: синтаксис Markdown для вставки с последующим управлением через HTML-тег `<img width="200">` или с помощью CSS-классов.

```
## Images

![This is an alt text.](/image/Markdown-mark.svg "This is a sample image.")
```



2.3.2 Внешние изображение

Вставка внешних изображений в Markdown осуществляется путём указания абсолютного URL-адреса изображения, размещённого в интернете, в стандартном синтаксисе: `![Альтернативный текст](https://ejemplo.com/imagen.jpg)`. Главное преимущество этого подхода — отсутствие необходимости хранить и синхронизировать файлы изображений в локальном проекте. Однако он создает критическую зависимость от доступности и стабильности внешнего ресурса. Если исходный сервер станет недоступен, изображение превратится в "битую ссылку". Кроме того, загрузка контента с чужих доменов может вызывать проблемы с производительностью, конфиденциальностью (трекеры) или блокировкой со стороны браузера (политика CORS). Для надежности в публичных или долгосрочных проектах рекомендуется загружать внешние изображения локально или использовать доверенные CDN. Для управления размером внешних изображений в Markdown также часто применяется встраивание HTML-тега `<img>` с атрибутами `width` и `height`.

Выполнения лабораторной работы

В качестве основной части задания вам необходимо освоить базовые команды для работы с Git, научиться компилировать шаблон отчета из формата `.md` в форматы `.pdf` и `.docx` с помощью `make`, а также изучить структуру исходного файла отчета.

Сначала откройте терминал и перейдите в рабочий каталог курса, используя команду ``cd ~/work/study/2025-2026/"Computer Architecture"/arch-pc/``. Затем обновите локальный репозиторий, чтобы получить последние изменения с сервера. Используйте команду ``git pull``.

Терминал сообщит, что все уже обновлено. Это означает, что ничего не было потеряно.

```
fernando@ubuntu25-10:~$ cd ~/work/study/2025-2026/"Архитектура компьютера"/arch-pc/
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git pull
Already up to date.
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$
```

Далее перейдите в каталог, содержащий шаблон отчета для этой лабораторной работы: ``cd labs/lab03/report``. Из этого каталога выполните команду ``make`` для сборки отчета. Процесс должен успешно завершиться, и в каталоге появятся два новых файла: ``report.pdf`` и ``report.docx``. Откройте оба файла, чтобы убедиться в их корректном отображении.

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ cd labs/lab03/report
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab03/report$ make
pandoc
to: latex
output-file: arch-pc--lab03--report.tex
standalone: true
self-contained: true
pdf-engine: xelatex
variables:
  graphics: true
  tables: true
default-image-extension: pdf
number-sections: true
toc: true
toc-depth: 2
cite-method: biblatex
metadata
documentclass: scrreprt
classoption:
  - DIVII
  - numbersnoendperiod
papersize: a4
header-includes:
  - \usepackage[options]{tableheading}
block-headings: true
lang: ru-RU
toc-title: Содержание
crossref:
  lof-title: Список иллюстраций
  lot-title: Список таблиц
  lol-title: Актинги
bibliography:
  - bib/cite.bib
csl: resources/csl/gost-r-7-0-5-2008-numeric.csl
colorlinks: false
lof: true
lot: true
font-size: 12pt
linestretch: 1.5
```

```
babel-lang: russian
babel-otherlangs: english
biblatexoptions:
  - backend=biber
  - languagexetras
  - autolangother*
csquotes: true
indent: true
author:
  name: Дмитрий Сергеевич Кулаков
  degrees: DSc
  orcid: 0000-0002-0077-7063
  email: kulyakov-ds@rudn.ru
  affiliation:
    - name: Российский университет дружбы народов
      country: Российская Федерация
      postal-code: 117198
      city: Москва
      address: ул. Милуто-Рыская, д. 6
  title: Учебно-отчет по лабораторной работе
  subtitle: Простейший вариант
  license: CC BY
pandoc
to: docx
output-file: arch-pc--lab03--report.docx
standalone: true
self-contained: true
default-image-extension: png
number-sections: true
toc: true
toc-depth: 2
variables: {}
metadata
lang: ru-RU
toc-title: Содержание
crossref:
  lof-title: Список иллюстраций
```

```
lot-title: Список таблиц
lol-title: Актинги
bibliography:
  - bib/cite.bib
csl: resources/csl/gost-r-7-0-5-2008-numeric.csl
author:
  name: Дмитрий Сергеевич Кулаков
  degrees: DSc
  orcid: 0000-0002-0077-7063
  email: kulyakov-ds@rudn.ru
  affiliation:
    - name: Российский университет дружбы народов
      country: Российская Федерация
      postal-code: 117198
      city: Москва
      address: ул. Милуто-Рыская, д. 6
  title: Учебно-отчет по лабораторной работе
  subtitle: Простейший вариант
  license: CC BY
Rendering PDF
running xelatex - 1
This is XeTeX, Version 3.141592653-2.6-0.999996 (TeX Live 2025/dev/Debian) (preloaded format=xelatex)
restricted \write18 enabled.
entering extended mode
generating bibliography
INFO - This is Biber 2.20
INFO - Logfile is 'arch-pc--lab03--report.blg'
INFO - Logfile is 'arch-pc--lab03--report.bcf'
INFO - Found 4 citekeys in bib section 0
INFO - Processing section 0
INFO - Looking for bibtext file 'bib/cite.bib' for section 0
INFO - LaTeX decoding ...
INFO - Found biblatex data source 'bib/cite.bib'
INFO - Overriding locale 'ru-RU' defaults 'variable = shifted' with 'variable = non-ignorable'
INFO - Overriding locale 'ru-RU' defaults 'normalization = NFD' with 'normalization = prenormalized'
INFO - Sorting list nty/global/global/global of type 'entry' with template 'nty' and locale 'ru-RU'
INFO - No sort tailoring available for locale 'ru-RU'
INFO - Writing 'arch-pc--lab03--report.bbl' with encoding 'UTF-8'
```

```
INFO - Output to arch-pc--lab03--report.bbl
running xelatex - 2
This is XeTeX, Version 3.141592653-2.6-0.999996 (TeX Live 2025/dev/Debian) (preloaded format=xelatex)
restricted \write18 enabled.
entering extended mode
running xelatex - 3
This is XeTeX, Version 3.141592653-2.6-0.999996 (TeX Live 2025/dev/Debian) (preloaded format=xelatex)
restricted \write18 enabled.
entering extended mode
Output created: _output/arch-pc--lab03--report.pdf
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab03/reports$
```

После проверки скомпилированных файлов удалите их с помощью команды `"make clean"`. Эта команда полезна для быстрого удаления ненужных файлов и сохранения в папке только исходных файлов. Убедитесь, что файлы `report.pdf` и `report.docx` были удалены.

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab03/report$ make clean
rm -rf _output
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab03/report$ git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
```

После того, как отчёт будет готов и опубликован в корневом каталоге проекта, сохраните все изменения в рабочей области и выполните три команды последовательно:

- ``git add``: Подготовьте все изменённые файлы к отправке.
- ``git commit -am 'feat(main): add lab-3 files'``: Зафиксируйте изменения с комментарием.
- ``git push``: Опубликуйте на GitHub.

Это было сделано без обращения в службу поддержки, и проблема была решена.

```
lab03/report$ git add .
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/
lab03/report$ git commit -am 'feat(main): add files lab-3'
[master a3a7409] feat(main): add files lab-3
 2 files changed, 1 insertion(+)
 create mode 100644 labs/lab03/report/report.md.save
 rename labs/lab03/report/{arch-pc--lab03--report.qmd => report.qmd} (100%)
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/
lab03/report$ git push
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 509 bytes | 509.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:AlexeiFRG1/study_2025-2026_arch-pc.git
 0a5db4f..a3a7409 master -> master
```

Выполнение самостоятельной работы

Задание 1

1. В соответствующем каталоге сделайте отчёт по лабораторной работе № 2 в формате Markdown. В качестве отчёта необходимо предоставить отчёты в трех форматах: pdf, docx и md.

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git add .
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git commit -am '
Добавление пдф, докс, мд'
[master 8b46185] Добавление пдф, докс, мд
 3 files changed, 330 insertions(+)
 create mode 100644 labs/lab02/report/НКАбд-02-25 _Ромеро _Алексей _2 _ЛР.docx
 create mode 100644 labs/lab02/report/НКАбд-02-25 _Ромеро _Алексей _2 _ЛР.md
 create mode 100644 labs/lab02/report/НКАбд-02-25 _Ромеро _Алексей _2 _ЛР.pdf
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git push

Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.20 MiB | 2.92 MiB/s, done.
Total 8 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:AlexeiFRG1/study_2025-2026_arch-pc.git
 a3a7409..8b46185 master -> master
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$
```

Задание 2

2. Загрузите файлы на github.

 НКАбд-02-25 _Ромеро _Алексей _2 _Л...	Добавление пдф, докс, мд
 НКАбд-02-25 _Ромеро _Алексей _2 _Л...	Добавление пдф, докс, мд
 НКАбд-02-25 _Ромеро _Алексей _2 _Л...	Добавление пдф, докс, мд

Выводы

В ходе этой лабораторной работы будут достигнуты следующие цели: освоение процедуры форматирования отчетов с использованием языка разметки Markdown; изучение его базового синтаксиса для форматирования текста, создания списков и вставки изображений; получение практического опыта работы с утилитой make для автоматической компиляции документов в форматах .pdf и .docx, а также закрепление навыков использования системы контроля версий Git для сохранения и фиксации результатов работы в удаленном репозитории.

Список литературы

Gruber, J. (2004). Markdown. Daring Fireball.

GitHub. (2023). Mastering Markdown. GitHub Guides.

CommonMark. (2023). Спецификация: Параграфы.

GitHub. (2023). Основы написания и форматирования: Заголовки.

CommonMark Spec. (2023). ATX-заголовки.

Gruber, J. (2004). Markdown: Синтаксис – Заголовки.

CommonMark Spec. (2023). Выделение и сильное выделение.

GitHub Guides. (2023). Основы форматирования: Стили текста.

Gruber, J. (2004). Markdown: Синтаксис – Выделение.

Gruber, J. (2004). Markdown: Syntax. Daring Fireball. В разделе "List" описаны базовые принципы создания списков.

Knuth, D. E. (1984). The TeXbook. Addison-Wesley

Oetiker, T., et al. (2021). The Not So Short Introduction to LaTeX2ε.

KaTeX Documentation. (s.f.). <https://katex.org/>

